



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms- Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, <u>Longest Common Subsequence</u> , 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours

BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos, Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K. Ahuja, Thomas L. Magnanti and James B. Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



Module 2: Design Paradigms: Dynamic Programming-Backtracking- Branch and Bound(10 Hours)

- **Dynamic Programming**
 - Matrix chain Multiplication
 - Assembly line Scheduling
 - **Longest Common Subsequence**
 - 0/1 Knapsack Problem
 - TSP- Travelling Salesman Problem
- **Backtracking**
 - N-queens Problem
 - Sum of Subsets
 - Graph Coloring
- **Branch and Bound**
 - LIFO and FIFO Methods
 - Job Selection Problem
 - 0/1 Knapsack Problem



Dynamic Programming

Dynamic programming is an algorithm design method that can be used when the solution to a problem can be viewed as the result of a sequence of decisions.

Principle of Optimality:

The principle of optimality states that an optimal sequence of decisions has the property that whatever the initial state and decision are, the remaining decisions must constitute an optimal decision sequence with regard to the state resulting from the first decision.



Longest Common Subsequence

- Given two sequences

$$X = (x_1, x_2, \dots, x_m)$$

$$Y = (y_1, y_2, \dots, y_n)$$

Find “Z” a maximum length common subsequence (LCS) of X and Y

- E.g.:

$$X = (A, B, C, B, D, A, B)$$

$$Y = (B, D, C, A, B, A)$$

- Subsequences of X:
 - A subset of elements in the sequence taken in order
(A, B, D), (B, C, D, B), etc.



Longest Common Subsequence

- **Problem Objective**

- Given two sequences **X** and **Y**, we say that a sequence **Z** is a common subsequence of X and Y if Z is a subsequence of both X and Y.
- For example, if **X = (A, B, C, B, D, A, B)** and **Y = (B,D,C,A,B,A)**, the sequence (B,C,A) is one of the common subsequence of both X and Y. The sequence (B,C,A) is not a longest common subsequence (LCS) of X and Y, however, since it has length 3 and the sequence (B,C,B, A) ,(B, D, A, B) , etc.... which is also common to both X and Y, has length 4.
- X and Y have no common subsequence of length 5 or greater.



LCS- Brute Force Solution

- For every subsequence of X , check whether it's a subsequence of Y
- There are 2^m subsequences of X to check
- Each subsequence takes $\Theta(n)$ time to check
 - Scan Y for first letter, from there scan for second, and so on.....
- Running time: $\Theta(n2^m)$



Recursive Algorithm to find actual LCS

LCS(i,j)

{

if (a[i] == '\0' || b[j] == '\0')

return 0;

else if(a[i]==b[j])

return 1 + LCS(i+1,j+1);

else

return max(LCS(i+1,j), LCS(i,j+1));

}



LCS- Optimal-substructure property

- What is Optimal Sub- Structure?
 - Solutions of sub problems are parts of the final solution.
- Given a sequence $\mathbf{X} = (x_1, x_2, \dots, x_m)$ we define the i -th prefix of X , **for $i = 0, 1, 2, \dots, m$** as **$\mathbf{X}_i = (x_1, x_2, \dots, x_i)$** .
- For example, if **$\mathbf{X} = (A, B, C, B, D, A, B)$** , then **$\mathbf{X}_4 = (A, B, C, B)$** and **$\mathbf{X}_0$** is the empty sequence.



LCS- Algorithm

- Finding the length of the LCS??
 - Define X_i, Y_j to be the prefixes of X and Y of length i and j respectively
- Define $c[i, j]$ to be the length of LCS of X_i and Y_j
- Then the length of LCS of X and Y will be $c[m, n]$

$$c[i, j] = \begin{cases} c[i-1, j-1] + 1 & \text{if } x[i] = y[j], \\ \max(c[i, j-1], c[i-1, j]) & \text{otherwise} \end{cases}$$



LCS- Recursive Solution

- Start with $i = j = 0$ (empty substrings of x and y)
- Since X_0 and Y_0 are empty strings, their LCS is always empty (i.e. $c[0,0] = 0$)
- LCS of empty string and any other string is empty, so for every i and j : $c[0,j] = c[i,0] = 0$
- Now consider the two cases:
- **First case:** $x[i] = y[j]$: Symbol in strings X and Y matches, so the length of LCS X_i and Y_j equals to the length of LCS of smaller strings **X_{i-1} and Y_{j-1} , plus 1**
- **Second Case:** $x[i] \neq y[j]$, As symbols don't match, our solution is not improved, and the length of $\text{LCS}(X_i, Y_j)$ is the same as before (i.e. **maximum of $\text{LCS}(X_i, Y_{j-1})$ and $\text{LCS}(X_{i-1}, Y_j)$**)

LCS- Finding Length Algorithm

LCS-Length(X, Y)

{

 m = length(X) // get the # of symbols in X

 n = length(Y) // get the # of symbols in Y

 for i = 1 to m c[i,0] = 0 // special case: Y_0

 for j = 1 to n c[0,j] = 0 // special case: X_0

 for i = 1 to m // for all X_i

 for j = 1 to n // for all Y_j

 if ($X_i == Y_j$)

 c[i,j] = c[i-1,j-1] + 1

 else

 c[i,j] = max(c[i-1,j], c[i,j-1])

 return c[m,n] // return LCS length for X and Y

}



LCS- Finding Length Algorithm

LCS-Length(X, Y)

$m = X.length$

$n = Y.length$

let $b[1..m, 1..n]$ and $c[0..m, 0..n]$ be new tables

for $i = 1$ **to** m

$c[i, 0] = 0$

for $j = 0$ **to** n

$c[0, j] = 0$

for $i = 1$ **to** m

for $j = 1$ **to** n

if $x_i == y_j$

$c[i, j] = c[i - 1, j - 1] + 1$

$b[i, j] = \nwarrow$

elseif $c[i - 1, j] \geq c[i, j - 1]$

$c[i, j] = c[i - 1, j]$

$b[i, j] = \uparrow$

else $c[i, j] = c[i, j - 1]$

$b[i, j] = \leftarrow$

return c and b



LCS Example – Step I

		j	0	1	2	3	4	5
			Y _j	B	D	C	A	B
i								
0	X _i							
1	A							
2	B							
3	C							
4	B							

$X = \text{ABCB}; \quad m = |X| = 4$

$Y = \text{BDCAB}; \quad n = |Y| = 5$

Allocate array $c[5,4]$

LCS Example- Step 2

		j	0	1	2	3	4	5
			Y _j	B	D	C	A	B
i								
0	X _i		0	0	0	0	0	0
1	A		0					
2	B		0					
3	C		0					
4	B		0					

for i = 1 to m c[i,0] = 0

for j = 1 to n c[0,j] = 0

LCS Example – Step 3

A B C B

B D C A B

		j	0	1	2	3	4	5
			Y _j	B	D	C	A	B
i								
0	X _i		0	0	0	0	0	0
1	A	0	0	0				
2	B	0						
3	C	0						
4	B	0						

if ($X_i == Y_j$)

$c[i,j] = c[i-1,j-1] + 1$

else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (Step 4)

ABCB
BDCAB

j	0	1	2	3	4	5
	Y _j	B	D	C	A	B
i	X _i					
0		0	0	0	0	0
1	A	0	0	0	0	
2	B	0				
3	C	0				
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (Step 5)

ABCB
BDCAB

j	0	1	2	3	4	5
	Y _j	B	D	C	A	B
i	X _i					
0		0	0	0	0	0
1	A	0	0	0	1	
2	B	0				
3	C	0				
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (Step 6)

ABCB
BDCAB

j	0	1	2	3	4	5
	Y _j	B	D	C	A	B
i	X _i					
0		0	0	0	0	0
1	A	0	0	0	1	1
2	B	0				
3	C	0				
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (Step 7)

ABCB
BDCAB

j	0	1	2	3	4	5
	Yj	B	D	C	A	B
i	Xi					
0	0	0	0	0	0	0
1	A	0	0	0	1	1
2	B	0	1			
3	C	0				
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (Step 8)

ABCB
BD CAB

j	0	1	2	3	4	5
i	Y _j	B	D	C	A	B
0	X _i	0	0	0	0	0
1	A	0	0	0	0	1
2	B	0	1	1	1	
3	C	0				
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (Step 9)

ABCB
BD CAB

j	0	1	2	3	4	5
i	Y _j	B	D	C	A	B
0	X _i	0	0	0	0	0
1	A	0	0	0	1	1
2	B	0	1	1	1	2
3	C	0				
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (Step 10)

ABCB

BD CAB

j	0	1	2	3	4	5
i	Y _j	B	D	C	A	B
0	X _i	0	0	0	0	0
1	A	0	0	0	1	1
2	B	0	1	1	1	2
3	C	0	1	1		
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (Step I I)

ABCB
BD CAB

j	0	1	2	3	4	5
i	Y _j	B	D	C	A	B
0	X _i	0	0	0	0	0
1	A	0	0	0	1	1
2	B	0	1	1	1	2
3	C	0	1	1	2	
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (Step 12)

ABCB
BDCAB

j	0	1	2	3	4	5
i	Y _j	B	D	C	A	B
0	X _i	0	0	0	0	0
1	A	0	0	0	1	1
2	B	0	1	1	1	2
3	C	0	1	1	2	2
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (Step I 3)

ABCB
BDCAB

j	0	1	2	3	4	5
	Yj	B	D	C	A	B
i	Xi					
0	0	0	0	0	0	0
1	A	0	0	0	1	1
2	B	0	1	1	1	2
3	C	0	1	1	2	2
4	B	0	1			

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (Step 14)

ABCB
BD CAB

j	0	1	2	3	4	5
i	Y _j	B	D	C	A	B
0	X _i	0	0	0	0	0
1	A	0	0	0	0	1
2	B	0	1	1	1	2
3	C	0	1	1	2	2
4	B	0	1	1	2	2

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (Step 15)

ABCB
BD CAB

j	0	1	2	3	4	5
	Y _j	B	D	C	A	B
i	X _i					
0		0	0	0	0	0
1	A	0	0	0	1	1
2	B	0	1	1	1	2
3	C	0	1	1	2	2
4	B	0	1	1	2	3

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Algorithm Running Time

- LCS algorithm calculates the values of each entry of the array $c[m,n]$
- So what is the running time?

$O(m*n)$

since each $c[i,j]$ is calculated in constant time, and there are $m*n$ elements in the array



How to find actual LCS ?

- So far, we have just found the *length* of LCS, but not LCS itself.
- We want to modify this algorithm to make it output Longest Common Subsequence of X and Y
- Each $c[i,j]$ depends on $c[i-1,j]$ and $c[i,j-1]$ or $c[i-1,j-1]$
- For each $c[i,j]$ we can say how it was acquired:

2	2
2	3

For example, here
 $c[i,j] = c[i-1,j-1] + 1 = 2+1=3$

How to find actual LCS ??

- Remember that

$$c[i, j] = \begin{cases} c[i-1, j-1] + 1 & \text{if } x[i] = y[j], \\ \max(c[i, j-1], c[i-1, j]) & \text{otherwise} \end{cases}$$

- Start from $c[m, n]$ and go backwards
- Look first to see if 2nd case above was true
- If not, then $c[i, j] = c[i-1, j-1] + 1$, so remember $x[i]$ (because $x[i]$ is a part of LCS)
- When $i=0$ or $j=0$ (i.e. we reached the beginning), output remembered letters in reverse order

Algorithm to find actual LCS

- Here's a recursive algorithm to do this:

```
LCS_print(x, m, n, c) {  
    if (c[m][n] == c[m-1][n]) // go up?  
        LCS_print(x, m-1, n, c);  
    else if (c[m][n] == c[m][n-1] // go left?  
        LCS_print(x, m, n-1, c);  
    else { // it was a match!  
        LCS_print(x, m-1, n-1, c);  
        print(x[m]); // print after recursive call  
    }  
}
```





Finding LCS

		j	0	1	2	3	4	5
			Y _j	B	D	C	A	B
i	Xi							
0			0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0	1	1	2	2	2
4	B		0	1	1	2	2	3

Finding LCS (2)

		j	0	1	2	3	4	5
i		Yj		B	D	C	A	B
	Xi							
0			0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0	1	1	2	2	2
4	B		0	1	1	2	2	3

LCS (reversed order): **B C B**

LCS (straight order): **B C B**
(this string turned out to be a palindrome)

Review: Dynamic programming

- DP is a method for solving certain kind of problems
- DP can be applied when the solution of a problem includes solutions to sub problems
- We need to find a recursive formula for the solution
- We can recursively solve sub problems, starting from the trivial case, and save their solutions in memory
- In the end we'll get the solution of the whole problem



Properties of a problem that can be solved with dynamic programming

- **Simple Subproblems**
 - We should be able to break the original problem to smaller subproblems that have the same structure
- **Optimal Substructure of the problems**
 - The solution to the problem must be a composition of subproblem solutions
- **Subproblem Overlap**
 - Optimal subproblems to unrelated problems can contain subproblems in common



Review: Longest Common Subsequence (LCS)

- Problem: How to find the longest pattern of characters that is common to two text strings X and Y
- Dynamic programming algorithm: solve sub problems until we get the final solution
- Sub problem: First find the LCS of *prefixes* of X and Y .
- This problem has *optimal substructure*: LCS of two prefixes is always a part of LCS of bigger strings



Conclusion

- Dynamic programming is a useful technique of solving certain kind of problems
- When the solution can be recursively described in terms of partial solutions, we can store these partial solutions and re-use them as necessary
- Running time (Dynamic Programming algorithm vs. Brute Force algorithm):
 - LCS: $O(m*n)$ vs. $O(n * 2^m)$
 - 0-1 Knapsack problem: $O(W*n)$ vs. $O(2^n)$





Thank You